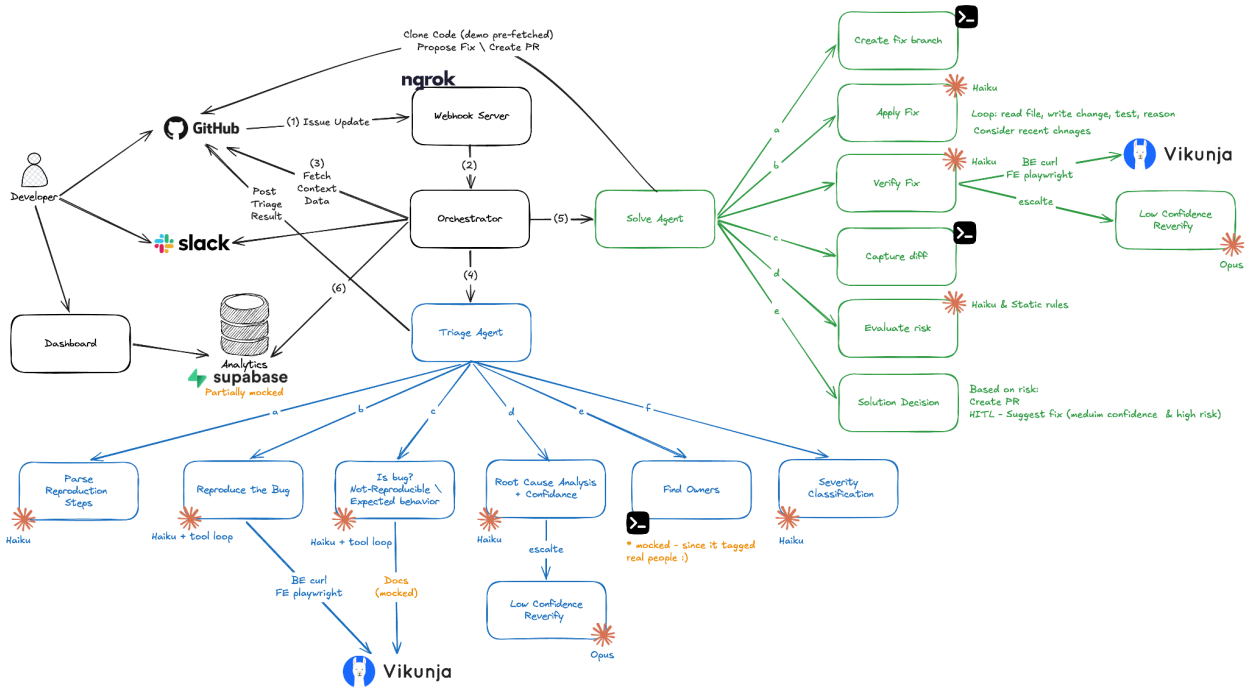


Port Technical Assignment: Agentic Bug Triage & Resolution

System Review

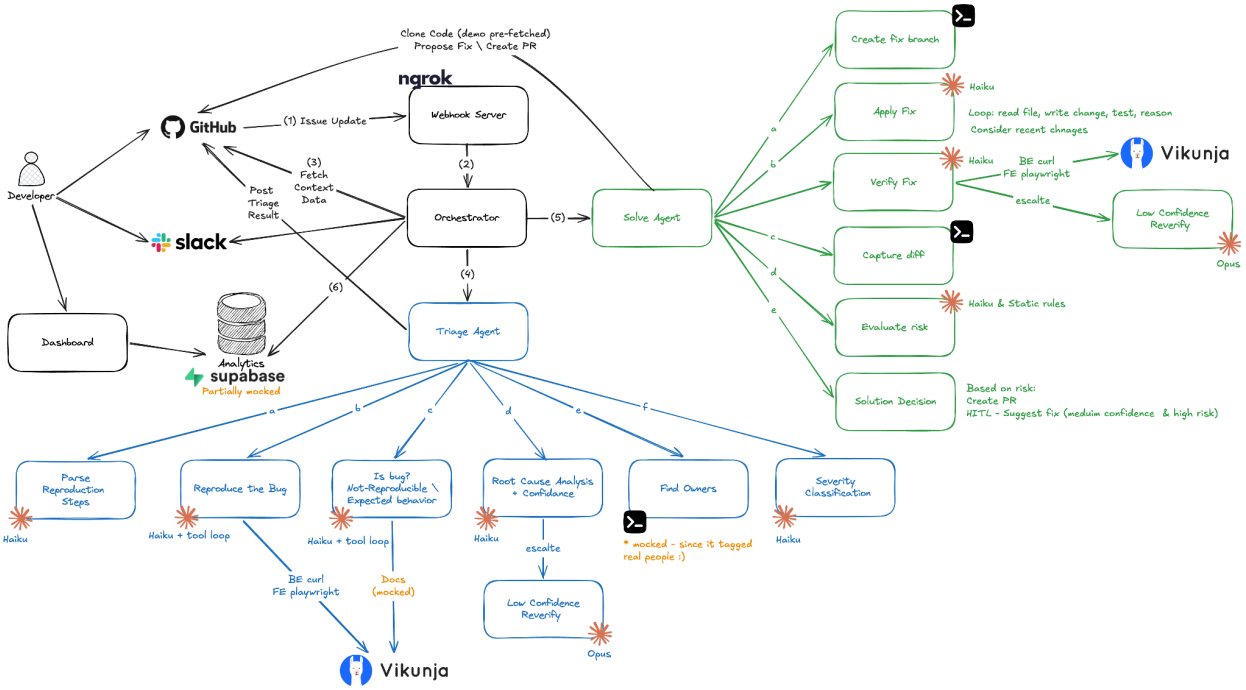
The system is built to triage and solve real issues reported on GitHub for a specific project.
It is deployed on Railway and listens for issues in [my Vikunja GitHub fork] (this deployment was done in the last few hours and has some limitations compared to what was presented in the demo, mainly sharing the before-and-after images).

Important Links

- [Project GitHub]
- [Demo recording]
- [Architecture overview image (attached at the bottom)]
- [My Vikunja fork GitHub]
- [Analytics Dashboard]

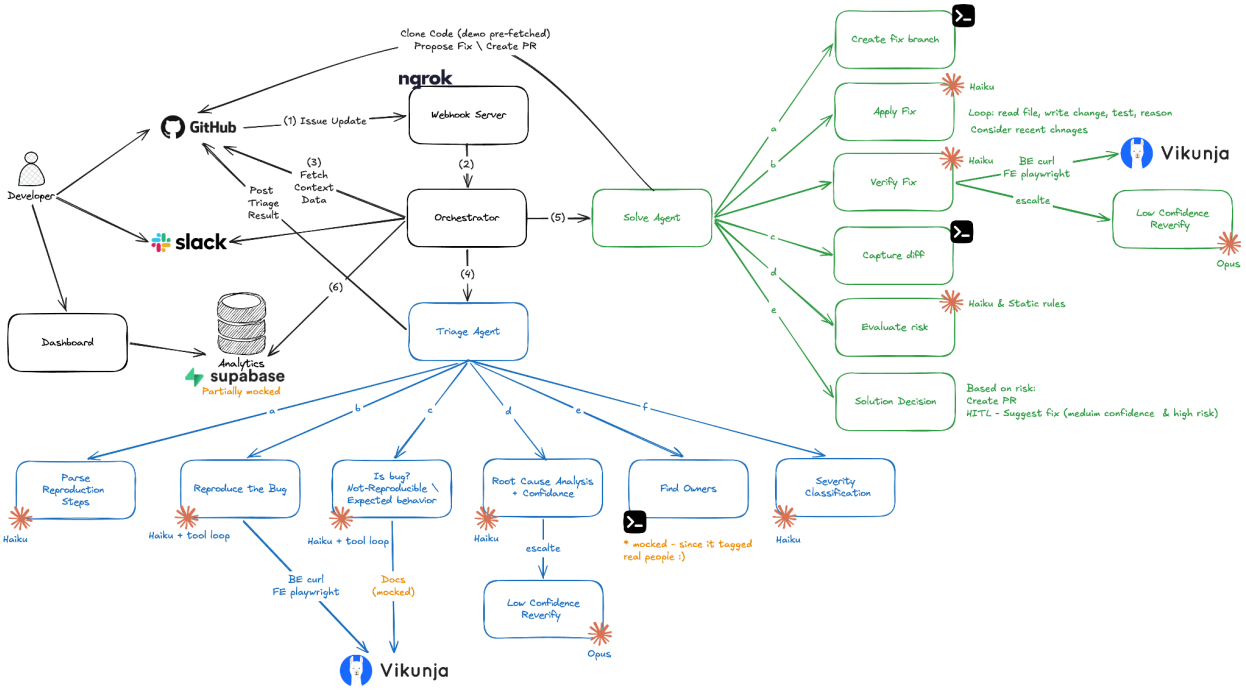
System Flow

1. Webhook server - connecting specific GitHub events, v1 locally using ngrok to trigger the system, v2 Railway doesn't require it.
2. **Orchestrator**:
    a. Fetching the context from GitHub and checking for deduplication (issue already handled).
    b. Call **Triage** agent:
        i. Parsing reproducing steps (Haiku).
        ii. Reproduce the bug (Haiku + tools) - limited to 25 iterations, will utilize curl for BE issues, and Playwright for FE ones.
            1. Check reproduction successfully - was the agent actually able to reproduce the bug (true/false, and *reproduction confidence score*). Inability to reproduce will cause the process to be stopped.



iii. Validate “is bug?” (Haiku + tools) - using the above and also the documentation for expected behavior (mocked, not reading the full Vikunja documentation).
iv. Root cause analysis (Haiku + tools) - identify the possible source of the issue and define ***certainty***.
    1. In case of low certainty, retry harder with Opus.
v. Find relevant people to tag (APIs) - experts who contributed the most, and the one who caused the bug - mocked (since it tagged real people 😂)
vi. Severity classification (Haiku) - Define the ***severity*** of the issue
vii. Comment posting to GitHub and notify Slack

c. Get back data, store in context
d. Call **Solve** agent:
    i. Create fix branch (CLI)
    ii. Develop and apply the fix (Haiku) - locate the issue, search previous commits (find possible bug introduction), make minimal changes, and verify by building and testing.
    iii. Verify - re-run the reproduction steps against the code, using curl for BE and utilize Playwright for FE. Define the ***verification confidence.***
        1. In case of low confidence, retry harder with Opus
    iv. Capture diff (CLI)
    v. Evaluate Risk (Haiku and rule-based) - define the ***risk*** of making the change.
        1. Make a level-of-autonomy decision: either generate a PR or wait for human feedback before making a PR (Human-in-the-loop, HITL).
            a. Low risk + non-CRITICAL severity - create PR
            b. Else - HITL
e. Evaluate the cost of the process using a tracker that counts tokens per model throughout the process.
f. Comment posting to GitHub and notify Slack
g. Update data for monitoring and observability - This will be available in the Dashboard for the agent developers

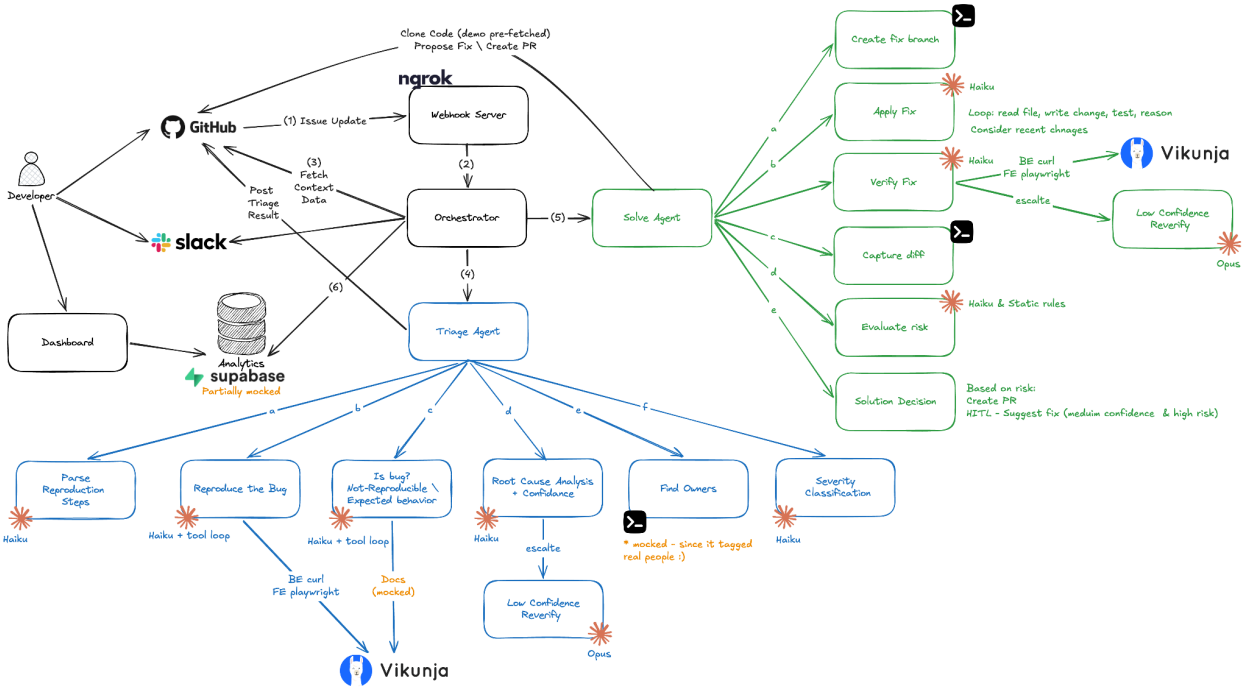


Basic Benchmarking

	BE Bug	BE with Opus	FE Bug	FE with Playwright
Avg. Time	90s	135s	73s	103s
Tokens-In	88k	108k Haiku 36k Opus	93k	200k
Tokens-Out	7k	5k Hiku 3k Opus	8k	6k
Avg. cost	$0.12	$0.4	$0.14	$0.23

Next Steps

1. Improve the system's remote deployment - This will allow handling scale and improving stability
    a. Implement cloning and on-demand code re-fetching from the repo.
    b. Improve the use of Playwright: post proof images and recordings to an image storage service and share them as links.
    c. Improve concurrency support - allowing multiple agents to work on multiple issues in parallel.
    d. Deploy on multiple instances in multiple regions.
2. Listening to more GitHub integrations and understanding the complete resolution/rejection cycle.
3. Add more data to the monitoring and improve the dashboards. Adding thresholds and alerts on system stability or performance degradations.
4. Connect to the documentation with MCP to identify expected behavior, not a bug.
5. Making the system learn -
    a. Index the codebase itself so triage finds relevant files via semantic search instead of hardcoded hints/grep.
    b. Index past resolved bugs so triage retrieves the most similar past fix
    c. Fallback handling of bad retrievals can mislead the model into copying the wrong fix
6. Better monitoring and handling of usage budget.



7. Extend the break conditions under which the models will decide not to make a fix, based on validation and certainty.
8. Better handling of secrets using a secure flow.
9. Improve the flow and UX both on GitHub and on Slack - improve quality of messages, and allow more actions using buttons.

link